

Metodología de Desarrollo de Software

Definición: Conjunto de pasos (PROCESO - QUE) y procedimientos (ACTIVIDADES - COMO) que deben seguirse para el desarrollo de software.

Otros aspectos:

- filosofías,
- reglas,
- técnicas, herramientas,
- documentación
- aspectos de formación

OBJETIVOS

- Mejores Aplicaciones.
- Proceso de Desarrollo (planificar y controlar los proyectos)
- Un Proceso Estándar en la organización.

ELEMENTOS

- Actividades y Tareas
- Procedimientos: forma de llevar a cabo las Tareas
- Comunicación entre Usuarios y Desarrolladores
- Productos: Intermedios o Finales
- Técnicas: como aplicar un Procedimiento + Gráficas y/o Textuales + Determinan el formato de los Productos de cada Tarea
- Herramientas Software: soporte a las Técnicas

TIPOS

1) CONVENCIONAL (50's)

- Desarrollo artesanal y ausencia de Metodología

- Enfocado en la Tarea de Programación
- Inconvenientes:
 - resultados impredecibles
 - No hay control sobre lo que está sucediendo en el Proyecto
 - Los cambios organizativos afectan negativamente al proceso de desarrollo (COMUNICACIÓN)
 - El éxito de los proyectos se basa mucho en la figura del “héroe”

2) ESTRUCTURADO: (60'S: ENTORNO ACADÉMICO, 70'S: INDUSTRIA)

- Académico: Normas para escribir código + Facilitar comprensión de Programas + Normas para la aplicación de estructuras de datos y de control
- Industrial: Mayor nivel abstracción (independencia del lenguaje programación) + Elemento básico de diseño: Módulo + Modularidad. Medidas de Calidad de Programas

Previo a eso:

- Descripción Narrativa Requisitos (Especificaciones): Monolíticas + Redundantes + Ambiguas + Imposibles de Mantener

Se propone:

- Especificaciones Funcionales: Gráficas + Particionadas + Mínimamente redundantes
- Modelos del sistema que representan: Los procesos + Los flujos + Las estructuras de los datos
- Enfoque Top-Down: Desde una visión general hasta un nivel de abstracción más sencillo
- Componentes Metodología De Diseño Estructurado De Yourdon:
 - PASO 1: Realizar los DFD (diagrama de flujo de dato) + DD (diccionario de datos) del sistema (análisis)
 - PASO 2: Realizar el diagrama de estructuras (Diseño)
 - PASO 3: Evaluar el diseño (Diseño)- acoplamiento y cohesión
 - PASO 4: Preparar el diseño para la implantación (Diseño)- dividir diseño para ser enviado a desarrollo

3) Desarrollo Orientado A Objetos (80's) - ADOO

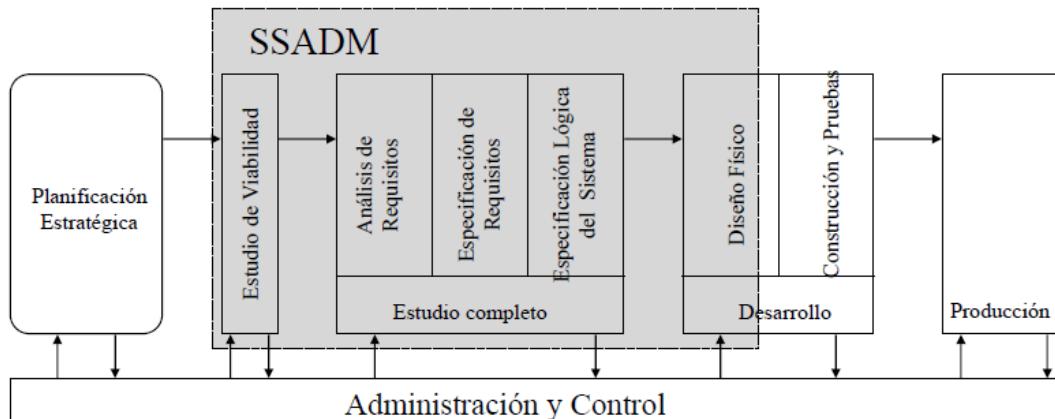
Definición: Modelado del Sistema examinando el dominio del problema como un conjunto de objetos que interactúan entre sí ---- Objetos: Encapsulan Comportamiento y Características

- Esencia: Identificación y organización de conceptos del dominio de la aplicación y no tanto de su representación final en un lenguaje de programación
- Trata Funcionalidad y Datos de forma conjunta.
- Principios: Abstracción + Ocultación de Información (Encapsulamiento) + Modularidad
- Influenciado por técnicas estructuradas

Otras Metodologías Importantes:

1) Structured Systems Analysis and Design Method - SSADM

- Creado para la Administración Pública del Reino Unido (1980)



Componentes:

- Estudio de viabilidad: soportar de manera rentable los requisitos comerciales.
- Análisis de requisitos:
 - se identifican los requisitos del sistema
 - se modela procesos llevados a cabo
 - estructuras de datos involucradas.

- Especificación de requisitos: se identifican requisitos funcionales y no funcionales detallados y se introducen nuevas técnicas para definir las estructuras de procesamiento y datos requeridas.
- Especificación del sistema lógico: se producen opciones de sistemas técnicos y el diseño lógico del proceso de actualización y consulta y los diálogos del sistema.
- Diseño físico: se crea un diseño de base de datos física y un conjunto de especificaciones de programa utilizando la especificación del sistema lógico y la especificación técnica del sistema.

Foco en análisis y diseño.

Novedad: incorporar el análisis de factibilidad

Problema de la metodología: definir base de datos antes del código

2) METRICA

- Versión 1 y 2 DEL 1989 al 1995 : estructurado
- Versión 3 - 2001: adaptable estructurado / OO
- Administración Pública España
- Consta de 3 procesos:
 1. Planificación de Sistemas de Información – (PSI)
 2. Desarrollo de Sistemas de Información:
 - a. Estudio de Viabilidad del Sistema (EVS)
 - b. Análisis del Sistema de Información (ASI)
 - c. Diseño del Sistema de Información (DSI)
 - d. Construcción del Sistema de Información (CSI)
 - e. Implantación y Aceptación del Sistema (IAS)
 3. Mantenimiento de Sistemas de Información (MSI)
- Tiene 4 Procesos De Soporte Y Organizativos:
 - Gestión de Proyectos,
 - Gestión de Configuración,

- Aseguramiento de Calidad (pruebas estáticas y dinámicas)
- Seguridad (gestión de riesgos técnicos)

3) MANTEMA

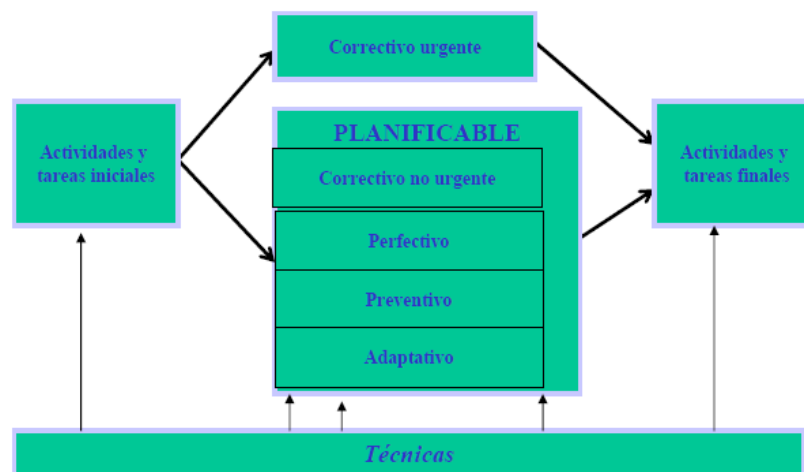
- especializada en Mantenimiento
- Desarrollado en Universidad de Castilla-La Mancha (1999)
- Propone taxonomía de actividades de mantenimiento:

1) No Planificable (NP):

- Correctivo Urgente (UC): localizar y eliminar los posibles defectos que bloquean el programa o los procesos de funcionamiento de la empresa.

2) Planificable (P):

- Correctivo No Urgente (NUC): localizar y eliminar los posibles defectos que no son bloqueantes.
- Perfectivo (PER): añadir al software nuevas funcionalidades solicitadas por los usuarios.
- Adaptativo (A): modificar el software para adaptarlo a cambios en el entorno de trabajo (hardware o software).
- Preventivo (PRE): modificar el software para mejorar sus propiedades (calidad, mantenibilidad, etc.).



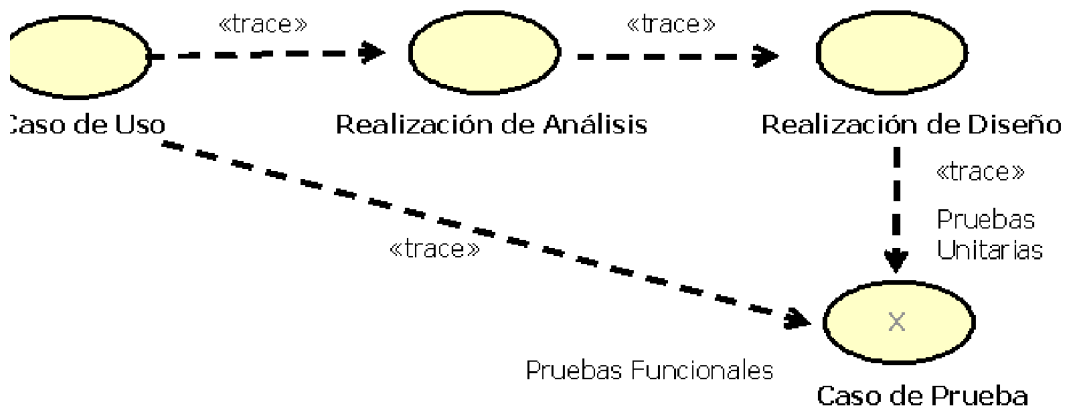
Novedad propuesta: manejo de tickets + análisis + lecciones aprendidas /indicadores

RUP: Rational Unified Process

Rational Software Corporation → junio del 1998 se lanza Rational Unified Process.

Características

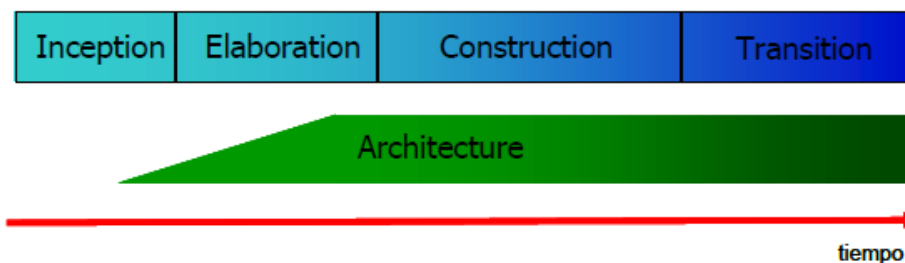
- Dirigido por Casos de Uso: captura de requisitos que fuerza a pensar en términos de importancia para el usuario y no sólo en términos de funciones. Los Casos de Uso representan los requisitos funcionales. Para RUP guían su diseño, implementación y prueba y proporcionan un hilo conductor permitiendo establecer trazabilidad entre los artefactos que son generados



- Proceso centrado en la arquitectura: establecimiento temprano de una buena arquitectura que no se vea fuertemente impactada ante cambios posteriores durante la construcción y el mantenimiento

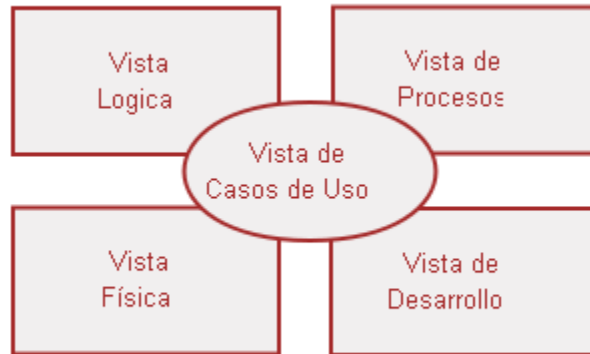
CU: Función / Arq: Forma

Tanto arquitectura como Casos de Uso deben evolucionar en paralelo durante todo el proceso de desarrollo de software



Propone VER el sistema desde diferentes perspectivas para comprender mejor el diseño por lo que la arquitectura se representa mediante varias vistas

Usa el Modelo 4+1 de la arquitectura, el cual recibe este nombre porque lo forman las vistas lógica, de implementación/development, de proceso y de físico/deploy, más la de Casos de Uso



1) Vista Lógica: estructura y funcionalidad. Se usan Diagramas de clase, comunicación y secuencia

2) Vista Desarrollo: punto de vista del programador. Se usan Diagrama de componentes y paquetes.

3) Vista Procesos: aspectos dinámicos. Los procesos y su comunicación. Tiempo de ejecución. Concurrencia, rendimiento y escalabilidad. Se usan Diagramas de Actividad.

4) Vista Física: componentes de software y conexiones. Se usan Diagrama de despliegue.

+1) VISTA DE CU: escenarios y funcionalidades. Se usan Diagramas de CU

Otras características:

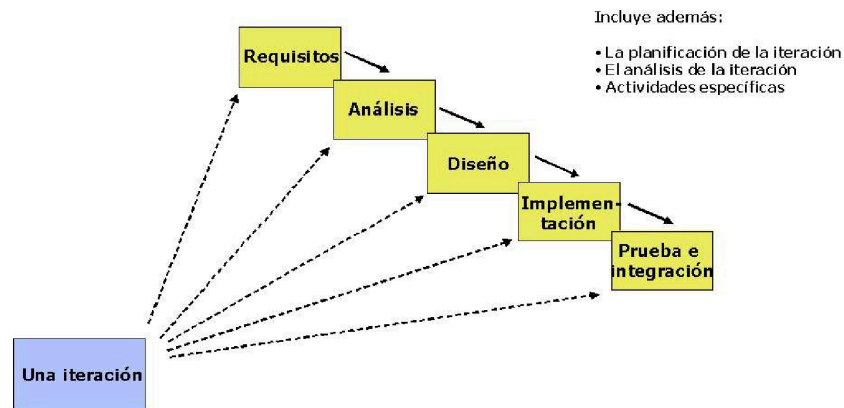
- Proceso iterativo e incremental: proceso iterativo e incremental en donde el trabajo se divide en partes más pequeñas o mini proyectos. Permitiendo que el equilibrio entre Casos de Uso y arquitectura se vaya logrando durante cada mini proyecto. Cada iteración se enfoca en una fase. NO es un proceso incremental DEL PRODUCTO, sino del resultado de cada fase.

Suele usar el prototipado operacional.

Requisitos, Análisis, Diseño, Implementación y Pruebas (mini cascada)

- + Planificación
- + Análisis de interacción
- + Integración

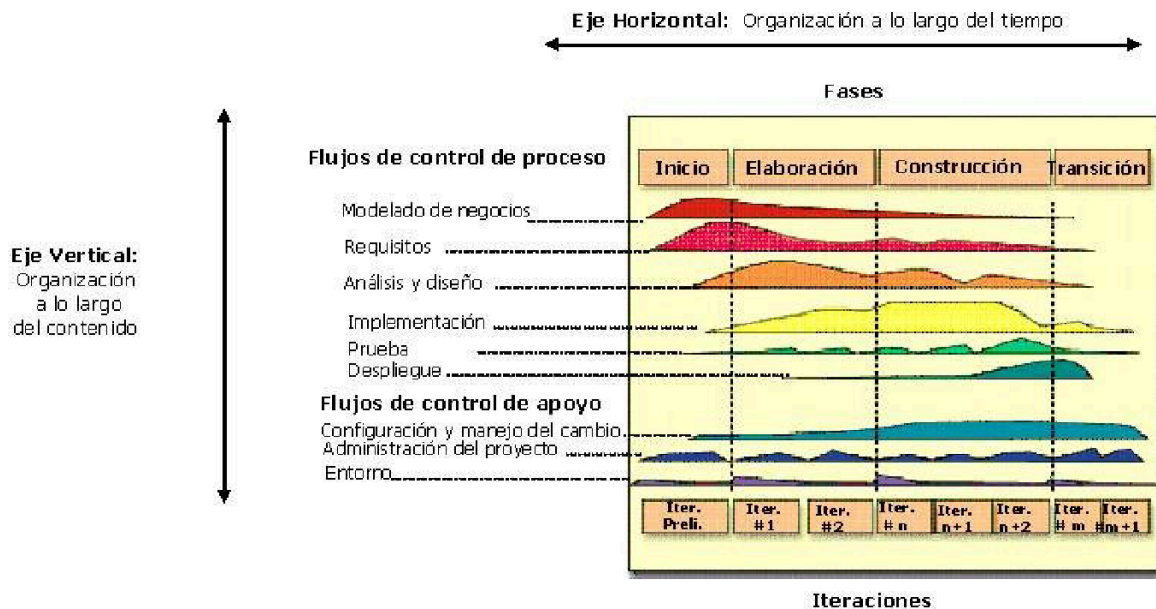
ITERACIÓN



- UML

Utilización de un único lenguaje de modelado: UML es adoptado como único lenguaje de modelado para el desarrollo de todos los modelos.

- Proceso:



Fases:

A) Inicio: (inception) modelado del negocio y de requisitos. Se identifican todos los actores y Casos de Uso, y se diseñan los Casos de Uso más esenciales (aproximadamente el 20% del modelo completo). Se desarrolla un plan de negocio

Resultados ppales:

- Un documento de visión: Una visión general de los requerimientos del proyecto, características clave y restricciones principales.
- Modelo inicial de Casos de Uso (10-20% completado).
- Un glosario inicial: Terminología clave del dominio.
- El caso de negocio.
- Lista de riesgos y plan de contingencia.
- Plan del proyecto, mostrando fases e iteraciones.
- Modelo de negocio, si es necesario
- Prototipos exploratorios para probar conceptos o la arquitectura candidata

Normalmente 1 iteración.

B) Elaboración: (elaboration) analizar el dominio del problema, establecer los cimientos de la arquitectura, desarrollar el plan del proyecto y eliminar los mayores riesgos.

Resultados ppales:

- Modelo de Casos de Uso completo al menos hasta el 80%: todos los casos y actores identificados, la mayoría de los casos desarrollados.
- Requisitos adicionales que capturan los requisitos no funcionales y cualquier requisito no asociado con un Caso de Uso específico.
- Descripción de la arquitectura software.
- Un prototipo ejecutable de la arquitectura.
- Lista de riesgos y caso de negocio revisados.
- Plan de desarrollo para el proyecto.
- Un manual de usuario preliminar (opcional).

N iteraciones.

C) Construcción: (construction) cada iteración se selecciona algunos Casos de Uso, se refina su análisis y diseño y se procede a su implementación y pruebas. Se realiza una pequeña cascada para cada ciclo

Resultados ppales:

- Modelos Completos (Casos de Uso, Análisis, Diseño, Despliegue e Implementación)
- Arquitectura íntegra (mantenida y mínimamente actualizada)
- Riesgos Presentados Mitigados
- Plan del Proyecto para la fase de Transición.
- Manual Inicial de Usuario (con suficiente detalle)
- Prototipo Operacional
- Caso del Negocio Actualizado

N iteraciones.

D) Transición: (transition) pretende garantizar que se tiene un producto preparado para su entrega a la comunidad de usuarios.

Resultados ppales:

- Prototipo Operacional
- Documentos Legales
- Caso del Negocio Completo
- Línea de Base del Producto completa y corregida que incluye todos los modelos del sistema
- Descripción de la Arquitectura completa y corregida
- Las iteraciones están dirigidas a conseguir una nueva versión

N iteraciones.

B) Organización ("cascada"):

- Modelado de negocios
- Requisitos
- Análisis y Diseño
- Implementación
- Prueba

- Despliegue

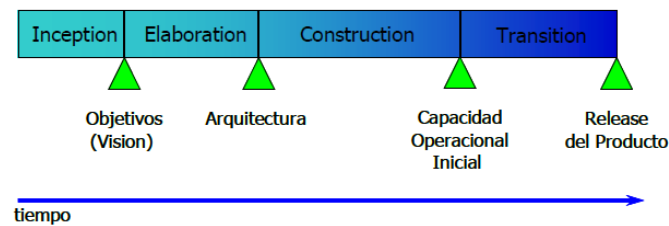
Otras:

- Gestión de la configuración y manejo del cambio

- Gestión del proyecto

- Entornos

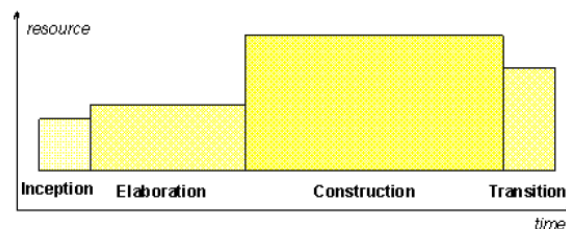
Cada fase se concluye con un hito bien definido, un punto en el tiempo en el cual se deben tomar ciertas decisiones críticas y alcanzar las metas clave antes de pasar a la siguiente fase, ese hito principal de cada fase se compone de hitos menores de cada iteración.



Duración vs Tiempo por FASE

	Inicio	Elaboración	Construcción	Transición
Esfuerzo	5 %	20 %	65 %	10%
Tiempo Dedicado	10 %	30 %	50 %	10%

Conformación y evolución del equipo de trabajo:



Pros:

- Primera metodología "seria" y robusta. Contempla todas las actividades de un proyecto

- Debería CUSTOMIZARSE

- Bueno (“bueno”) para grandes proyectos

Contras:

- Sigue confiando en el método predictivo. Sigue funcionando a base de cascada.
- Malo para proyectos pequeños. Pesado
- Propietario

Prince2

- Origen y Definición: PRINCE2 (PRejects IN Controlled Environments) es una metodología de gestión de proyectos desarrollada en el Reino Unido, ampliamente adoptada a nivel mundial. Su enfoque se centra en la organización, gestión y control de proyectos.
- Principales Características: Define roles y responsabilidades claros, procesos estructurados y la importancia de la justificación continua del negocio durante todo el ciclo de vida del proyecto.
- Inicio de Proyecto: Definición de los objetivos del proyecto, el equipo y la organización. Se crea el acta de constitución del proyecto.
- Dirección de Proyecto: Tomar decisiones estratégicas, aprobar fases y resolver problemas escalados. Se mantiene la justificación del negocio.
- Control de Fase: Monitoreo y control de la ejecución de una fase del proyecto. Se gestionan riesgos y problemas.
- Justificación Continua del Negocio: Asegura que el proyecto sigue siendo viable y rentable a lo largo de su ciclo de vida.
- Gestión de Riesgos: Identificación, evaluación y control de riesgos que puedan afectar al proyecto.
- Calidad: Definición y control de los estándares de calidad requeridos para cumplir con los objetivos del proyecto.

ÁGILES vs PREDICTIVAS

Predictivas:

- define con detalle cuál es el producto de software previsto a desarrollar
- elabora un plan de desarrollo a partir del cual se derivan o calculan costos y duraciones
- Durante la ejecución se realizan actividades de seguimiento y control para evitar o controlar lo más posibles desvíos sobre lo planificado
- planificación se apoya en el supuesto de tener claramente definido el objetivo o la solución a construir de antemano

- el éxito de este enfoque está basado en una correcta especificación del objetivo del proyecto durante la definición del proyecto y las actividades preliminares para determinar su alcance.

Ágil:

- tiene como meta entregar el producto con el mayor valor posible para el cliente en un tiempo dado
- Se usa para eso un ciclo de construcción iterativo (secuencia de etapas cortas que llevan a la entrega de una porción del producto terminada) e incremental (en cada iteración se entrega más funcionalidades)
- permite ir descubriendo cuales son las funcionalidades que aportan más valor al cliente a medida que se entregan las porciones del producto en cada iteración, e incorporar cambios proveniente del feedback del cliente sobre el uso real del software

Ágiles: manifiesto ágil

- Se firma el 12 de Febrero del 2001, en Snowbird, Utah, EEUU
- se juntaron 17 personas, y decidieron asociar el adjetivo agile a sus metodologías, para asociarles una noción de resultados rápidos y liviandad.



1) Individuos e interacciones sobre procesos y herramientas

- No se niega la necesidad de procesos y herramientas: deben ser una ayuda para guiar el trabajo.
- Deben adaptarse a la organización, a los equipos y a las personas; y no al revés
- Sin personas con el conocimiento y una actitud adecuada, los procesos y las herramientas no generan resultados

2) Software funcionando sobre documentación exhaustiva

- Conceptualmente la documentación no aporta valor directo al negocio.
- El valor para el negocio: software funcionando en un ambiente operativo.
- Manifiesto Agile reconoce que al fin y al cabo la única medida real de avance de un proyecto es la entrega de software funcionando
- ver un software funcionando facilita el feedback del cliente o futuro usuario en comparación con la lectura de un documento de especificación de requerimientos por ejemplo.

3) Colaboración con el cliente sobre negociación contractual

- suelen fracasar los modelos contractuales cerrados con procedimientos de gestión de cambios muy definidos (identifican si la culpa de los retrasos la tiene el proveedor o el cliente)
- sumar al cliente como un miembro más del equipo, que se integra y colabora diariamente en el grupo de trabajo.
- marco contractual de alto nivel sobre el cual se construye una relación de confianza basada en los resultados logrados.

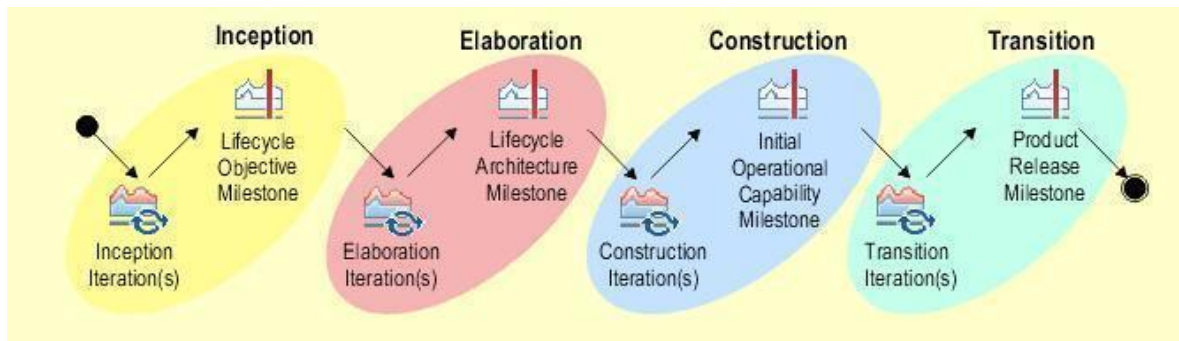
4) Respuesta ante el cambio sobre seguir un plan

- planificación y control para evitar desviaciones sobre el plan suele fracasar
- se enfoca en la anticipación y la adaptación a través de mecanismos de feedback constantes, de incorporación del cambio y de replanificación continua.

OpenUP

- Eclipse Foundation. IBM dona BUP (Basic UP, una adaptación de RUP para pequeños equipos). Se vuelve gratuita
- Mantiene desarrollo iterativo, orientado en CU y escenarios, centrado en la arquitectura y gestión de riesgos
- Se han excluido la mayoría de las partes opcionales de RUP y se han fusionado muchos elementos.

- Equipos de 3 a 6 personas e involucran de 3 a 6 meses de esfuerzo de desarrollo.



Fases:

- Inicio. Objetivos: Entender qué construir. Identificar funcionalidad Clave. Determinar al menos una posible solución. Entender costos, calendario y riesgos del proyecto.
- Elaboración. Objetivos: mayor nivel de detalle de los requerimientos. Diseñar, implementar y validar la línea base arquitectónica. Mitigar riesgos y lograr estimaciones de costos y calendarios más precisos.
- Construcción. Objetivos: Iterativamente desarrollar un producto completo que pueda ser transicionado a la comunidad usuaria. Minimizar los costos de desarrollo y lograr cierto nivel de paralelismo.
- Transición. Objetivos: Realizar Beta Testing para determinar si se alcanzaron las expectativas de los usuarios. Alcanzar la concordancia con los stakeholders de que el producto está terminado. Mejorar la performance futura a través del análisis retrospectivo del proyecto.

Pros:

- metodología ágil. centrado al cliente (prototipos) y con iteraciones cortas.
- Se puede adaptar con otros procesos (otras áreas, no solo desarrollo)
- Ya que es apropiado para proyectos pequeños y de bajos recursos permite disminuir las probabilidades de fracaso en los proyectos pequeños e incrementar las probabilidades de éxito.
- Permite detectar errores tempranos a través de un ciclo iterativo.

- Evita la elaboración de documentación, diagramas e iteraciones innecesarios requeridos en la metodología RUP.

Contras:

- A veces omite contenido que puede ser de interés en el proyecto.
- Se espera que cubra un amplio sistema de necesidades para los proyectos de desarrollo en un plazo muy corto.
- Al ser una metodología de bajo formalismo existirá la posibilidad, si no se tiene cuidado, de que el proyecto pueda perder rumbo debido a la desorganización

Essential UP (EssUP)

- Elegir aquellas prácticas que sean aplicables a su situación y combinarlas en su propio proceso: mejora con respecto a RUP
- noviembre de 2005, Ivar Jacobson (co-creador de RUP y UML)
- integración de prácticas eficaces de entre los tres principales campos de proceso: el proceso unificado, los métodos ágiles y el proceso de madurez (mejora de procesos).
- mismos conceptos base que RUP (CU y arquitectura), pero “ágil” desde el equipo y con mejora de procesos
- diferencia con RUP: las prácticas no están encadenadas
- 8 prácticas esenciales
 - Prácticas técnicas:
 - Arquitectura: obtener una base firme para el desarrollo de sistemas de alta calidad y robustos. Artefactos: Documentación de la arquitectura, Casos de prueba sobre la arquitectura, Vistas de la arquitectura, Verificación de la arquitectura de forma incremental.
 - Iterativa: desarrollo incremental. Artefactos: El backlog con cambios, cosas que no funcionan correctamente o fallos, y trabajo por ser realizado. El plan de proyecto (número y tipo de las iteraciones)
 - Caso de uso. Artefactos: CU de especificación de los requisitos, escenarios y casos de prueba. La generación de pruebas
 - Componente. Desarrollar complejos sistemas formados de componentes más pequeños y más simples. Artefactos: Un diseño del sistema a implementar, identificando el conjunto de componentes requeridos. Una descripción de cada componente, incluyendo su comportamiento e

interfaces. Código fuente y unidades de prueba para componentes. Construcciones integradas del sistema del componente y las pruebas y los resultados para verificar las construcciones.

- Producto. Administrar el desarrollo de sucesivas evoluciones de un sistema software como una serie de versiones del producto. Artefactos: El caso de negocio para establecer el valor del producto. El análisis de los stakeholders para asegurar que ellos comprenden y están involucrados en el proyecto. La lista de capacidades, glosario y descripción de cada versión para definir el producto y sus versiones. El plan de proyecto

- Prácticas transversales: proceso, equipo (ágil), modelado

- Un conjunto de cartas, cada una de las cuales describe una práctica. Esto se debe a que Ivar Jacobson cree que la gente compra sus libros pero pocos los leen (pobre Ivar...)

XP

- Extreme Programming (o XP) define un conjunto de valores, principios y prácticas para el desarrollo rápido de software de alta calidad.

- busca maximizar el valor creado para el cliente en un plazo lo más corto posible.

- extremo en el sentido que lleva buenas prácticas conocidas del desarrollo de software a su extremo (razonable) para maximizar sus efectos positivos sobre los proyectos de desarrollo.

- El primer proyecto XP fue iniciado en 1996, y luego XP demostró ser exitoso en múltiples organizaciones de distintos tamaños y distintas industrias en el mundo entero.

- Los fundadores de XP son Kent Beck, Ward Cunningham y Ron Jeffries (los 3 firmastes en el Manifiesto)

- se enfoca en la satisfacción del usuario, en lugar de entregar todo lo que podría desear al usuario en una fecha lejana del futuro, el proceso entrega el software que se necesita en el momento necesitado.

- enfatiza también el trabajo colaborativo, permitiendo al equipo su auto-organización para resolver de la mejor forma los problemas que ocurren.

- Valores:

- Comunicación:

- Todos los involucrados son parte del equipo y se comunican cara a cara diariamente.

- Se trabaja en conjunto en todo, desde los requerimientos hasta el código.
- Se crea la mejor solución posible a los problemas en conjunto.
- Simplicidad:
 - Se hace lo necesario y lo pedido, pero nada más. Esto permite maximizar el valor creado para la inversión realizada hasta el momento.
 - Se busca hacer pequeños pasos simples hacia el objetivo final y mitigar las fallas a medida que ocurren.
 - Se produce un resultado que genera el orgullo del equipo y que se puede mantener a largo plazo con costos aceptables.
- Feedback:
 - El equipo busca y consigue feedback de su producción testeando su software desde el primer día.
 - El equipo entrega el software al cliente lo antes posible e implementa los cambios sugeridos.
- Respeto:
 - Cada uno da y siente el respeto merecido por su aporte de valor como miembro del equipo.
 - Los desarrolladores respetan la idoneidad del cliente y recíprocamente.
- Coraje:
 - El equipo dice la verdad sobre el avance y las estimaciones del proyecto.
 - No se documentan excusas sobre las fallas porque se planifica el éxito.
 - No hay miedos porque se trabaja en conjunto.
 - El equipo se adapta a los cambios de requerimientos y de tecnologías cuando ocurren

Las 12 prácticas de XP

1. Cliente In-Situ

- disponibilidad del cliente, no solo para ayudar el equipo de desarrollo, sino también como parte del equipo.

- Todas las fases de un proyecto XP requieren comunicación e interacción con el cliente, y preferentemente cara a cara en el mismo sitio.
- Un cliente real debe sentarse con el equipo y estar disponible para contestar preguntas, resolver conflictos y definir prioridades a bajo nivel.
- tener una o varias personas del cliente asignadas al equipo de desarrollo, y es aconsejable que sean expertos que tienen o van a tener un uso real del software en desarrollo.
- se persigue que sean personas con la autoridad y las habilidades para proveer la información adecuada del sistema a desarrollar y para tomar las decisiones necesarias sobre los requerimientos y las prioridades en el momento oportuno.

2. Semana de 40 Horas

- Los proyectos de desarrollo tradicionales suelen necesitar horas extras y esfuerzos heroicos para entregar el software en la fecha prometida.
- El resultado suele ser un equipo agotado y menos productivo, lo cual genera una fuerte baja en la calidad del software producido, ya sea en el corto, mediano o largo plazo.
- Horas extras y esfuerzos heroicos son señales de problemas mayores, como por ejemplo compromisos por encima de las posibilidades, estimaciones pobres, falta de recursos, y otros.
- XP asegura que el equipo de desarrollo no se compromete con más trabajo que el que demostraron ser capaces de completar.
- La cantidad de trabajo a la cual el equipo se puede comprometer en la iteración actual es la misma cantidad de trabajo que el equipo terminó con éxito en la iteración anterior.
- Este compromiso siempre se hace al principio de la iteración y con el acuerdo completo del cliente.
- El resultado lógico es una mayor calidad. Como el equipo es dueño de sus propias estimaciones (no hay un líder de proyecto que impone las estimaciones, como suele pasar en otros tipos de enfoque de gestión), suele tener más confianza para terminar el trabajo en el tiempo comprometido.
- El estrés de los miembros del equipo suele reducirse ya que logran gestionar mejor su propia planificación.

3. Uso de Metáforas

- una persona que trata de explicar un concepto a otra intenta encontrar una referencia común a los dos que permita representar el concepto en una forma entendible por ambos.
- identificar una metáfora (o varias) que permita al equipo sintetizar el diseño esencial del software en un concepto que entiendan todos.
- Es una herramienta que permite eliminar los problemas de comunicación que suelen aparecer por las diferencias de perspectivas o de background de cada uno.
- Una metáfora no puede ser perfecta. Puede ser necesario utilizar varias y no esperar que las similitudes permitan abarcar todos los aspectos del sistema a desarrollar.
- Por ejemplo, metáfora de "un camaleón" que explica cómo una ventana cambia de transparencia.

4. Diseño Simple

- utilizar el diseño lo más simple posible que permita cumplir con la necesidad.
- Es probable que los requerimientos cambien en el futuro, con lo cual se propone hacer únicamente lo necesario para cumplir con los requerimientos actuales
- Para explicar lo que se entiende con "simple", la idea es que se construya lo suficiente para poder seguir agregando funcionalidades, pero sin anticiparse en que podría ser necesario en el futuro.
- El uso de patrones de diseño también permite conseguir simplicidad a través del uso de construcciones estándares y enfoques que fueron probados

5. Refactoring

- Martin Fowler define el refactoring como "el proceso de cambiar un software de tal forma que no afecte el comportamiento externo del código pero que sí permita mejorar su estructura interna."
- Podría interpretarse como mejorar la cohesión interna del desarrollo.
- Un ejemplo de refactoring en programación orientada a objetos es llevar una porción de código duplicado a un método donde podrá ser reutilizado con un simple llamado al método, en lugar de tenerlo replicado varias veces en toda la aplicación.
- actualizar y evolucionar el diseño inicial constantemente a través del refactoring. La idea es que el refactoring sea una preocupación constante del equipo de desarrollo para

asegurar que el diseño del sistema evoluciona correctamente y que pueda ser mantenido a largo plazo.

6. Pair Programming (Programación de a Pares)

- técnica de desarrollo de software en la que dos programadores trabajan en la misma tarea de codificación.
- Los programadores comparten una misma estación de trabajo.
- Se intercambian los roles de conductor y navegador con frecuencia.
- El conductor escribe el código, mientras que el navegador revisa el código en tiempo real.
- El navegador ofrece sugerencias y planifica los siguientes pasos.

7. Entregas Cortas

- hacer entregas tempranas y frecuentes, agregando unas pocas funcionalidades cada vez.
- La práctica apunta a un ciclo de desarrollo iterativo corto e incremental.
- valor concreto para el negocio rápidamente.
- cuando más temprano los usuarios pueden empezar a utilizar el software, más van a poder recibir un retorno sobre la inversión en el desarrollo. También este mecanismo permite a los usuarios dar un feedback temprano basado en una experiencia real de uso del software.
- Las entregas cortas ayudan también al cliente a tener confianza en el progreso del proyecto.
- Se busca llegar a una entrega productiva cada semana o cada dos semanas, donde al final de cada iteración se consigue un software testeado y funcionando.

8. Pruebas

- Pruebas de Unidad (Unit Tests) en base a TDD
 - testear la funcionalidad de pequeñas porciones de código (por ejemplo una clase o un método).
- Pruebas de Aceptación

- deben ser definidas por el cliente para asegurar que la aplicación funciona correctamente.

- son pruebas de alto nivel de la aplicación, más orientadas a la operación real del negocio.

- Una funcionalidad es considerada terminada cuando todos los tests de aceptación correspondientes fueron ejecutados con éxito.

- se recomienda automatizar los casos de prueba de aceptación, si no es posible se sugiere que exista una lista de pasos a seguir para ejecutar la prueba manualmente.

- Las pruebas de aceptación revelan inconvenientes por alguna sutileza respecto a la definición de una funcionalidad. Con esta práctica los desarrolladores pueden atacar estos problemas tempranamente y hacer los cambios necesarios inmediatamente.

9. Estándares de Código

- cada miembro del equipo escribe código con los mismos estándares: no se debería poder identificar quién del equipo ha escrito una porción específica del código solo leyéndolo

- El estándar de código es un conjunto de reglas que fueron acordadas y que el equipo se compromete a respetar a lo largo del proyecto, se suele llamar también "convenciones".

- El estándar suele especificar para un lenguaje de programación elegido:

- un estilo para el código fuente

- un formato para el código fuente

- algunos patrones y construcciones prohibidos para evitar la aparición de potenciales defectos

- algunos patrones y construcciones recomendados para manejar situaciones conocidas con buenas prácticas acordadas

- El efecto es el de facilitar la lectura del código producido por otra persona y mejorar la mantenibilidad del mismo.

10. Propiedad Colectiva

- evitar que una sola persona sea "dueña" de un módulo de la aplicación

- Se espera de todos los desarrolladores que puedan trabajar sobre cualquier porción del código en cualquier momento.
- La propiedad colectiva del código implica que cada uno es responsable de todo el código, y también que cada uno tiene el permiso de modificar cualquier porción del código.
- genera la necesidad de transferir conocimiento del código a todo el equipo e incrementa la comunicación entre todos.
- La programación de pares facilita esta transferencia.
- Beneficio: cuando un defecto es detectado en una porción del código, cualquier miembro del equipo lo puede solucionar.
- También permite a un equipo poder seguir con el desarrollo aun cuando un miembro del equipo falta o se va del equipo.

11. Integración Continua

- práctica de automatizar la integración de los cambios de código de varios contribuidores en un único proyecto de software
- se integra periódicamente
- se testea periódicamente

12. Juego de Planificación

- el equipo de desarrollo y el cliente cooperan para producir el máximo valor para el negocio de la forma más rápida posible.
- Suele ocurrir en una reunión al inicio de la iteración a la cual participa el equipo de desarrollo y el cliente. La dinámica es la siguiente:

1. El cliente presenta una lista de las funcionalidades deseadas para el sistema.

- Cada funcionalidad es escrita en formato de Historia de Usuario, que da un nombre a la funcionalidad y describe brevemente el comportamiento requerido.

2. El equipo de desarrollo estima cuánto esfuerzo de desarrollo es necesario para cada Historia de Usuario.

- se define el esfuerzo total que el equipo puede producir durante la iteración.

- el equipo divide cada Historia de Usuario en tareas técnicas asignables a una sola persona

- luego son estimadas por el mismo equipo de desarrollo.

3. El cliente decide qué Historias de Usuario implementar y en qué orden

Dinámica de un proyecto con XP:

- todos los desarrolladores están ubicados físicamente en una misma habitación, usualmente sentados alrededor de una larga mesa en el medio de la misma.

- Los equipos XP trabajan en una serie de ciclos con iteraciones fijas.

- Una iteración típicamente dura entre una y tres semanas según el equipo, pero la duración es siempre la misma.

- Al inicio de cada iteración, el equipo se reúne con el cliente para una reunión de planificación. En esta reunión, repasan las funcionalidades que el cliente quiere terminadas en esta iteración, dividiendo cada funcionalidad en tareas técnicas de una persona. Luego a cada desarrollador se le asigna tareas específicas y las estima. Ningún desarrollador puede asignarse más trabajo en esta iteración que lo que terminó en la iteración anterior.

- Durante el resto de la iteración, el equipo va a implementar las funcionalidades comprometidas, trabajando de a pares sobre todo el código producido.

- Para toda porción de código, los desarrolladores primero crean tests unitarios antes de escribir el código y se integra todo el código producido una vez por día, en general al finalizar la jornada laboral.

- El cliente provee tests de aceptación para validar las funcionalidades que el equipo está desarrollando.

- Al final de la iteración (usualmente un viernes), los desarrolladores entregan un software funcionando al cliente. El software puede no estar completo, pero toda la funcionalidad entregada funciona completamente, sin defectos.

- El cliente acepta/rechaza la entrega

- El lunes siguiente se reúnen nuevamente para planificar la iteración siguiente, y el ciclo se repite.

Lean Software Development:

- Principios:

- Eliminar los desperdicios
- Ampliar el aprendizaje
- Decidir lo más tarde posible
- Reaccionar tan rápido como sea posible
- Potenciar el equipo
- Crear la integridad
- Verse todo el conjunto

TDD

- técnica de desarrollo de software que usa iteraciones de desarrollo cortas basadas en casos de prueba escritos previamente que definen las mejoras deseadas o nuevas funcionalidades.
- requiere que los desarrolladores creen pruebas unitarias automatizadas para definir los requisitos del código antes de escribir el código en sí mismo
- Las pruebas contienen afirmaciones que son verdaderas o falsas.
- El objetivo es escribir código claro que funcione. Ejecutar las pruebas rápidamente confirma el comportamiento correcto a medida que los
- es un método de diseño de software, no sólo un método de pruebas.
- está relacionado con los primeros conceptos de pruebas de programación de XP, en 1999, pero más recientemente se está creando un interés general mayor en sí mismo.

Proceso

1. Agregar una prueba

- cada nueva funcionalidad empieza con la escritura de una prueba
- Esta prueba deberá fallar inevitablemente porque se escribe antes de que se haya implementado la característica.

- Para escribir una prueba, el desarrollador debe tener y entender claramente la especificación y los requisitos de la característica.
- El desarrollador puede llevar a cabo esto a través de casos de uso e historias de usuario que cubran los requisitos y las condiciones de excepción.
- No hace falta que sea una nueva funcionalidad, puede implicar también una variación o modificación de una prueba existente.

2 Ejecutar las pruebas y comprobar que la última que se ha añadido falla

- Esto valida que las pruebas están funcionando correctamente y que las nuevas pruebas no pasan erróneamente, sin la necesidad de código nuevo.

3 Escribir algo de código, realizar cambios en la implementación

- El siguiente paso es escribir algo de código que haga que se puedan pasar las pruebas.
- El código escrito en esta etapa no será perfecto y puede, por ejemplo, pasar la prueba de una forma poco elegante. En pasos sucesivos se mejorará y perfeccionará.
- Es importante resaltar que el código escrito sólo se diseña para pasar la prueba; no se debería predecir más funcionalidades.

4 Ejecutar las pruebas automatizadas y ver que tienen éxito

- Si todas las pruebas ahora pasan, el programador puede estar seguro de que el código cumple todos los requisitos probados.

5 Refactorizar el código para mejorar su diseño

- Ahora se puede limpiar el código si es necesario.
- Volviendo a ejecutar las pruebas, se puede estar seguro de que la refactorización no ha dañado ninguna funcionalidad existente.
- Repetir el ciclo con una nueva prueba.

Ventajas

- se definen de manera formal los requisitos que se espera que cumpla la aplicación.

- Los casos de prueba sirven como documentación del sistema.
- se piensa en la forma correcta de utilizar un módulo que aún no existe.
- Las pruebas permiten perder el miedo a realizar modificaciones en el código, ya que tras realizar modificaciones se volverán a ejecutar los casos de pruebas para comprobar si se ha cometido algún error.

Inconvenientes

- difícil de usar en situaciones donde hacen falta todas las pruebas funcionales para determinar éxito o fracaso (interfaces de usuario, conexiones con bases de datos y algunos que dependen de configuraciones de red específicas)
- Sin la creencia de toda la organización de que TDD va a mejorar el producto, la gestión sentirá que se pierde tiempo escribiendo pruebas.
- Las pruebas se han visto históricamente como una posición más baja que los desarrolladores o arquitectos (status quo)

KANBAN

- (Kan) “visual” + (Ban) “insignia” o “sello”. es una tarjeta pero también el proceso
- Uso En Industria Tradicional:
 - objetivo: limitar el trabajo en curso (WIP) para asegurar un flujo de producción continuo y optimizado, sin espera o sobreproducción que lleven a trabajo de inventario o sobre-stock.
 - cada pieza en producción o terminada lleva un kanban
 - existen en Proceso ANTERIOR y el Proceso POSTERIOR
 1. Se utiliza un número fijo y limitado de tarjetas Kanban
 2. El proceso posterior solamente retira partes terminadas del proceso anterior cuando las necesita.
 - el proceso anterior no “empuja” partes terminadas al proceso posterior, sino que es el proceso posterior que “retira” las partes terminadas del proceso anterior cuando las necesita.
 - El “depósito” (Store) es el lugar donde las partes terminadas son ubicadas, y sirve de cola del trabajo entre los dos procesos

- Cuando un trabajador del proceso posterior busca en el depósito una nueva parte terminada, también se devuelve un Kanban (señal visual) de producción, que sirve de instrucción de producción para el proceso anterior. Este mecanismo es fundamental para que el proceso anterior nunca produzca partes sin necesidad explícita del proceso posterior.

- La tarjeta Kanban es adjuntada físicamente a las partes, recolectada y re-utilizada: el número de tarjetas Kanban disponible sea fijo y limitado.

- pueden tomar forma de tarjetas sobre las cuales se encuentra información como el identificador o nombre de la parte, cantidad, tipo de pallet, ubicación del depósito, etc.

- Tipos de KANBAN:

- Kanban para retirar: es el ítem que permite al trabajador del proceso posterior retirar una parte terminada en el depósito del proceso anterior.

- Kanban para producir: provee al proceso anterior la instrucción de producir una nueva parte terminada para el proceso posterior.

Reglas:

1. El cliente (proceso posterior) retira partes en la cantidad exacta definida por el Kanban
2. El proveedor (proceso anterior) produce partes en la cantidad exacta y secuencia especificada por el Kanban
3. Ninguna parte es creada o movida sin un Kanban
4. Un Kanban debe acompañar cada ítem en todo momento
5. Nunca se mandan defectos o cantidades incorrectas al próximo proceso posterior
6. La cantidad de Kanbans es reducida con precaución para limitar inventarios y revelar problemas de flujo

Primer uso en IT

- 2004: David Anderson introdujo en el equipo de mantenimiento continuo para una Unidad de Negocio en Microsoft, cuya situación y rendimiento era al principio bastante por debajo de otros equipos parecidos y lejos de cumplir con las distintas expectativas involucradas.

- Conclusiones al respecto según David Anderson:

- Kanban permitió cambios incrementales en la forma de trabajo, con pocos riesgos políticos y con una resistencia al cambio mínima.
- Kanban puso a la luz oportunidades de mejoras que no implicaban cambios complejos en los métodos de ingeniería de software empleados.
- Generó:
 - incremento de la productividad de más de 300%
 - una reducción de 90% del tiempo promedio de un ciclo completo de desarrollo
 - mejora de alrededor de 90% de la predictibilidad del trabajo del equipo.
- Los cambios introducidos pueden requerir mucho tiempo para generar su efecto completo.
- En este caso se logró el efecto completo en 15 meses.

LOS 3 PRINCIPIOS DE KANBAN

- mostrar el proceso
- limitar el wip
- optimizar el proceso

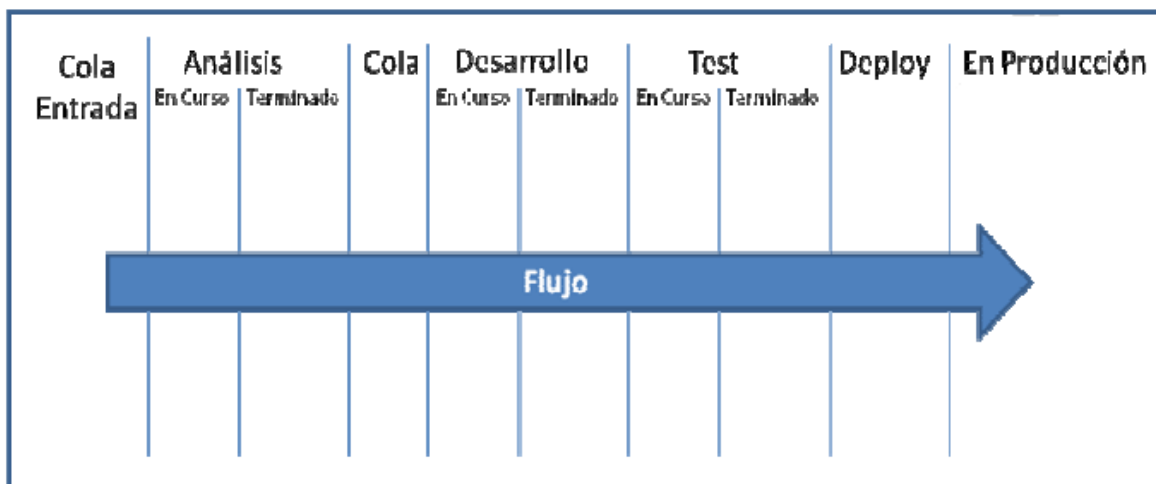
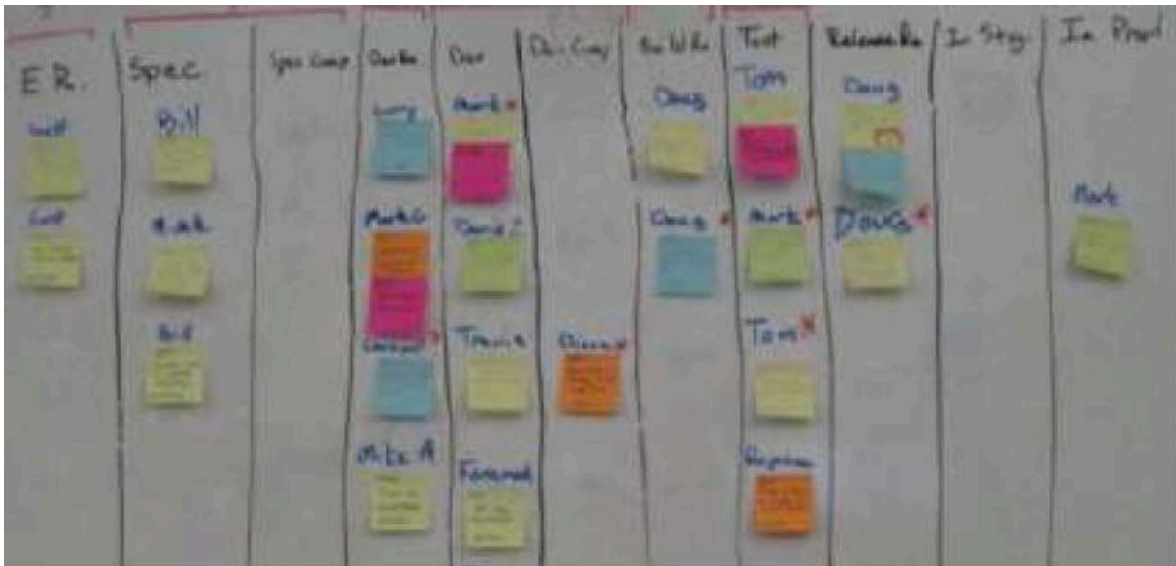
1. Mostrar el Proceso

- visualizar los ítems de trabajo y el proceso de trabajo en un tablero físico o herramienta digital equivalente.

Creación del Tablero

- expuesto en la sala donde trabaja el equipo.
- flujo del proceso de creación de valor de izquierda a derecha, dibujando columnas para cada actividad de la secuencia definida.
- se pueden dibujar dos sub-columnas para distinguir trabajo en curso y trabajo terminado. Se agregan columnas correspondientes al punto de entrada y de salida del sistema Kanban.
- Finalmente se agregan las columnas correspondientes de buffer o cola entre actividades.

- Luego ubicar en estas columnas las tarjetas de los ítems de trabajo en curso, en forma de post-its o papel, con la información que mencionamos para cada ítem, como en el ejemplo a continuación:



- De esta forma logramos visualizar la secuencia de actividades del proceso de creación de valor, el estado de avance de los distintos ítems de trabajo en curso, quien está asignado a cada ítem y otra información de gestión de mucha utilidad

2. Limitar el Trabajo en Curso

- acordar entre los involucrados límites sobre la cantidad de ítems de trabajo que pueden encararse a la vez.

- Limitar el WIP permite reducir el multi-tasking (trabajar en varias tareas a la vez), lo cual tiene efectos importantes:

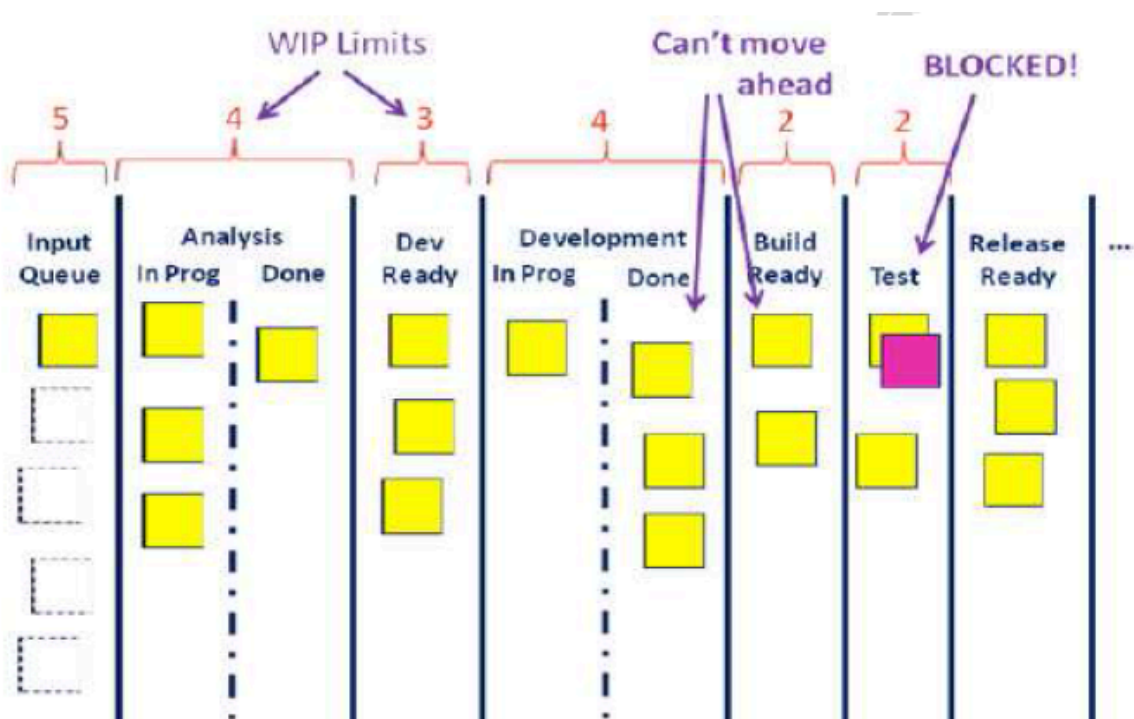
 - 20% del tiempo se pierde en el cambio de contexto por tarea, son lo cual tener menos tareas a la vez reduce esta pérdida de tiempo

 - Ejecutar tareas secuencialmente genera resultados más rápido.

- Límites de WIP:

 - Un límite de WIP provoca una restricción sobre cuántos ítems de trabajo pueden estar en el mismo paso del proceso de creación de valor (en cada columna del tablero).

 - Únicamente cuando un ítem de trabajo progresa en el proceso, se abre un espacio para que un nuevo ítem de trabajo pueda entrar en el paso correspondiente.



Explicitar los límites de WIP:

- es fundamental que todos los límites de WIP sean conocidos y acordados entre todos los involucrados.

- En particular es necesario que el negocio, el cliente, los usuarios, el management, los responsables de los procesos anteriores y posteriores y todo el equipo de desarrollo estén de acuerdo con los límites elegidos y que sepan las consecuencias y los mecanismos que eso implica.

- ¿Qué límites de WIP elegir?

- estos límites suelen calibrarse mejor a medida que el equipo encuentra su ritmo óptimo de trabajo a través de distintas mejoras y prueba distintos límites hasta encontrar unos convenientes.

- En consecuencia no es necesario acordar demasiado importancia a la definición inicial de estos límites, sino que como sugerencia puede ser mejor elegir rápidamente unos límites de WIP que parezcan lógicos y poner el foco en revisar periódicamente si estos límites se tienen que ajustar (por arriba o por abajo).

- Se suele utilizar un promedio con un valor de entre uno a tres por persona.

- Limitar la cola de entradas:

- Acordar un límite de tamaño sobre la cola de entradas al sistema Kanban (la primera columna del tablero) permite hacer foco sobre los próximos ítems de trabajo a encarar por el equipo y en particular genera un proceso de priorización basado en el valor para el negocio de cada ítem.

- Si la cola de entrada es por ejemplo limitada a sólo dos ítems de trabajo a la vez, el negocio/cliente/usuario va a tener que priorizar periódicamente los dos próximos ítems que quiere ver implementados en el software.

- Suele simplificar la priorización tener un límite de cola de entrada bajo, y en general provoca que el esfuerzo periódico de priorización de los ítems sea bajo.

3. Optimizar el Flujo

- monitorear y seguir el avance de los ítems de trabajo

- Objetivos

- Lograr un flujo de desarrollo estable, previsible y adecuado a las necesidades del negocio.

- El foco principal para habilitar compromiso y confianza con un sistema Kanban.

- Si un ítem de trabajo parece estar frenado, el equipo debería debatir para resolver como se puede rehabilitar el flujo de nuevo.

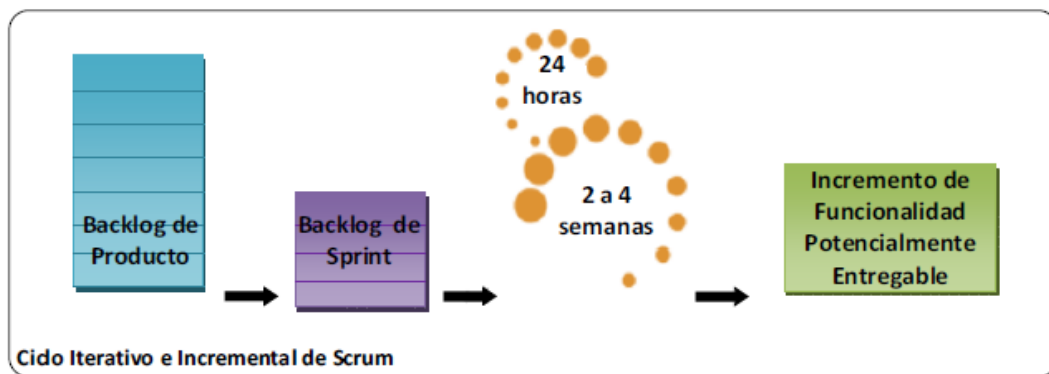
- Si todos los ítems de trabajo de un paso en particular parecen estar frenados, puede haber un cuello de botella en el sistema, y el equipo deberá también debatir (eventualmente incluyendo roles de management) para cambiar el proceso de creación de valor y solucionar el problema.

Conclusiones:

- "Lo importante no es tu proceso, lo importante es tu proceso para mejorar tu proceso"
- define muy pocas reglas a seguir, pero estas pocas reglas tienen un potencial muy fuerte para llevar a la mejora continua de los procesos de trabajo.
- la esencia de Kanban al principio es cambiar lo mínimo posible.
- evitar cambiar el proceso, los cargos, los roles y responsabilidades y las prácticas específicas de trabajo cuando se implementa Kanban.

SCRUM

- Jeff Sutherland creó el proceso de Scrum en el 1993
- utilizó el término "scrum" de una analogía que compara equipos de alto rendimiento y multi-disciplinario a la formación del "scrum" de los equipos de rugby.



1

- Un Dueño del Producto crea una lista priorizada de requisitos llamada Backlog de Producto.

- Durante la Planificación del Sprint, el Equipo selecciona una pequeña porción de esta lista, un Backlog de Sprint, y decide cómo implementar estos requisitos. El equipo tiene un

tiempo limitado, un Sprint, para terminar su trabajo - usualmente dos a cuatro semanas - pero se junta cada día para constatar su progreso (Reunión Diaria).

- En este camino, el Scrum Master mantiene el equipo enfocado hacía su objetivo.

Al final del sprint, el trabajo debería ser Potencialmente Entregable, o sea estar listo para enviar a un cliente, empaquetar para su distribución o presentar a un sponsor.

- El sprint termina con una Revisión de Sprint y una Retrospectiva.

- Cuando se inicia el próximo sprint, el equipo selecciona otra pequeña porción del backlog de producto y empieza a trabajar de nuevo. Este ciclo se repite hasta que el backlog de producto se haya completado, que el presupuesto se haya gastado o que un deadline haya llegado, según la especificidad de cada proyecto. No importa lo que genera el fin del trabajo, Scrum asegura que se haya terminado el proyecto de mayor valor al final del proyecto

SPRINT

- Sprint es el nombre de la iteración en Scrum

- duración fija para todos los sprints, en general entre una y cuatro semanas.

- se ejecutan uno tras el otro respetando está duración, sin tiempo muerto entre el sprint que termina y el sprint que empieza.

- El objetivo es de transformar un conjunto acordado de ítems del Backlog de Producto en un Incremento de Funcionalidad de Producto Potencialmente Deployable

ROLES EN SCRUM

- DUEÑO DE PRODUCTO

- es la voz del cliente, el que establece una visión sólida del producto a desarrollar y priorizar continuamente los requisitos para alcanzar dicha visión.

- es parte del equipo

- canalizar las necesidades del negocio para el producto a desarrollar.

- tiene la autoridad final

- tiene que asegurarse que el equipo entiende la visión del producto y sus distintos requisitos.

- responsabilidad: maximizar el valor para el negocio respecto a la inversión

- responsabilidad: inspeccionar y adaptar el producto

- SCRUM MASTER

- facilitador y agente de cambio para la aplicación adecuada de Scrum en el equipo y la organización (asegurar que el equipo, el Dueño de Producto y la organización adhieran a los valores, prácticas y reglas de Scrum)

- facilitador neutro con respecto a los resultados

- lleva al equipo a través del coaching a ser más productivo y a generar productos de mejor calidad.

- eliminar los impedimentos que frenan el trabajo del equipo (trabajar para removerlos y asegurar las mejores condiciones posibles de trabajo para el equipo)

- proteger al equipo de potenciales interferencias proveniente de la organización que pueden afectar al trabajo del equipo.

- EQUIPO

- grupo auto-organizados, multi-disciplinario (todos poseen todas las habilidades necesarias para resolver los problemas planteados por el Dueño de Producto) y con autonomía que desarrolla el producto.

- responsabilidad: transformar el Backlog de Producto en incrementos de funcionalidad potencialmente entregable en cada sprint.

- perfiles: programación, pruebas, análisis funcional, arquitectura, diseño de interfaz gráfica o diseño de base de datos.

- el equipo es responsable en su conjunto de la entrega de incrementos de funcionalidades

- promover el aprendizaje y perfeccionamiento de todos los miembros del equipo de todas las tareas involucradas en el desarrollo.

- auto-autogestión: de ninguna manera el Scrum Master o el Dueño de Producto le explica al equipo como transformar el Backlog de Producto en incrementos de funcionalidad potencialmente entregable.

- El equipo tiene autonomía para definir cómo se debe cumplir este objetivo y cómo organizar el trabajo de todos sus miembros para hacerlo de la mejor manera posible, pidiendo claridad y apoyo del Dueño de Producto cuando es necesario.

- El tamaño recomendado de equipo es de 7 personas más o menos dos.

- Cuando son menos de cinco, hay menos interacciones y como consecuencia se gana menos productividad con Scrum.

- Si hay más de nueve personas, se requiere demasiada coordinación y se puede generar demasiada complejidad en las actividades grupales de Scrum.

- los roles de Dueño de Producto y Scrum Master no están incluidos en esta cuenta, al menos que también tengan un rol como miembro del equipo, trabajando sobre tareas del Backlog de Sprint.

ARTEFACTOS DE SCRUM

- BACKLOG DE PRODUCTO

- Definición: es una lista única, pública y dinámica que recopila una secuencia priorizada de requerimientos para el producto. Algunos de estos requerimientos tienen una estimación de alto nivel de esfuerzo o complejidad. Se suele decir que el Backlog de Producto representa el "Qué" esperado del producto, sin preocuparse por el "Como"

- Historias de Usuario: forma de escribir un ítem del Backlog de Producto que se enfoca en la necesidad de los involucrados de entender como la funcionalidad se va a usar.

- BACKLOG DE SPRINT

- Definición: conjunto reducido negociado de ítems del Backlog de Producto que el equipo se compromete a completar durante el tiempo alocado al Sprint.

- Cada ítem incluido en el sprint es dividido en tareas detalladas que el equipo va a completar, con una estimación de duración que no supere un día de esfuerzo.

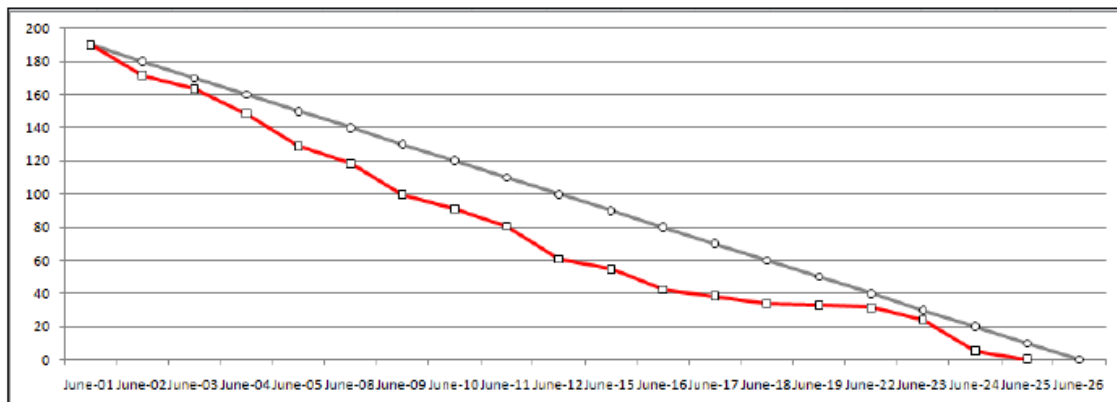
- El Backlog de Sprint es actualizado diariamente por el equipo, y muestra en cada momento las tareas pendientes, en curso y terminadas para completar los ítems del sprint, así como una estimación del esfuerzo pendiente de cada tarea no terminada y a quién está asignada la tarea.

- Se suele utilizar un Tablero de Sprint como forma de visualizar el Backlog de Sprint, o algún otro soporte herramental que permite darle visibilidad.

- Diagrama de Burndown

- forma gráfica de mostrar el avance de un sprint

- registrar diariamente el esfuerzo restante para completar todos los ítems comprometidos para el sprint en curso.
- cada día se suma el esfuerzo restante de todas las tareas en estado Pendiente y En Curso, una vez que el equipo haya actualizado todos los esfuerzos restantes.
- Esta suma es la cantidad de horas necesarias de parte del equipo para que se pueda completar el sprint.
- muestra en el eje X el tiempo, con un rango que representa todos los días hasta terminar el sprint actual (por ejemplo diez días hábiles si es un sprint de dos semanas de duración)
- El eje Y representa el esfuerzo restante del equipo para completar todos los ítems comprometidos, con una graduación en horas de esfuerzo restante.
- Existe una línea de progreso teórico que une el esfuerzo restante al principio del sprint con el valor de esfuerzo restante cero al final del sprint.
- Es el progreso teórico que un equipo debería seguir para tener un ritmo constante y continuo para llegar a cumplir con el sprint si las estimaciones fueran perfectas y todas las tareas identificadas.
- Cada día el Scrum Master irá marcando para el día correspondiente el esfuerzo restante actualizado. El objetivo es llegar a un esfuerzo nulo antes del final del Sprint.



- En este ejemplo vemos un sprint de 20 días de duración, con un esfuerzo restante estimado inicial de 190 horas.
- En gris se ve la línea de progreso teórico y en rojo el registro diario del esfuerzo restante actualizado
- Si la pendiente sube, se puede ver que el esfuerzo restante está creciendo. Puede pasar cuando se están agregando tareas no previstas, o que surgen problemas que hacen que las tareas duren más de lo estimado.

- Si la pendiente es horizontal de un día al otro, quiere decir que el esfuerzo restante no cambió de un día al otro, o sea que los eventuales avances fueron compensados por incrementos en esfuerzos restantes estimados (nuevas tareas, complicaciones en tareas existentes, etc.)

- Si la pendiente pasa por arriba de la línea de progreso teórico, se puede interpretar como que el equipo se esté atrasado y que no se pueda cumplir con los objetivos del sprint siguiendo este ritmo.

- Si la pendiente pasa por debajo de la línea de progreso teórico, se puede interpretar que el equipo está avanzando más rápido que lo previsto (puede ser que se eliminaron algunas tareas, que algunas tareas se terminaron en menos tiempo que lo previsto, etc.)

Tener esta suma actualizada diariamente permite tener una visión muy clara del avance del equipo, así como una forma gráfica de monitorear el ritmo del equipo y detectar tempranamente eventuales problemas de ritmo.

CEREMONIAS DE SCRUM

- PLANIFICACIÓN

1) ¿Cuándo?: Al iniciar el sprint, es la primera actividad de cada sprint.

2) Participantes: Dueño de Producto + Scrum Master + Equipo

3) Objetivos

- Que el Equipo de Desarrollo entiendan los ítems más prioritarios del Backlog de Producto

- Que el Equipo de Desarrollo y el Dueño de Producto acuerden el conjunto de ítems del Backlog de Producto que se van a desarrollar durante el próximo sprint

- Que el Equipo de Desarrollo defina las tareas requeridas para construir las funcionalidades acordadas durante el próximo sprint

4) Duración: entre dos y ocho horas.

5) Artefactos Involucrados

- Backlog de Producto: se usa para entender y elegir los ítems a comprometer para el sprint y se actualiza durante la reunión

- Backlog de Sprint: se define durante la reunión

6) Dinámica

1. Entendimiento y Selección de Ítems del Backlog de Producto

1. El Dueño de Producto presenta los ítems más prioritarios del Backlog del Producto.

2. El Equipo se asegura de haber entendido correctamente el alcance y los detalles principales de cada ítem, así como los criterios para la aceptación de este ítem por parte de Dueño de Producto. Deben hacer todas las preguntas necesarias para lograr este entendimiento preliminar.

3. El Equipo determina una estimación de alto nivel de cada uno de los ítems presentados. Se suelen usar técnicas de estimaciones relativas, como Planning Poker o T-Shirt Sizing.

4. En función de estas estimaciones y tomando en cuenta la capacidad del equipo, el equipo define un alcance del sprint identificando los ítems que se compromete terminar.

5. Eventualmente el Dueño de Producto puede pedir ajustes a este alcance y/o negociar algunos cambios

2. Definición del Backlog de Sprint

1. Para cada ítem seleccionado para el sprint, el equipo identifica todas las tareas necesarias para llegar al incremento de funcionalidad potencialmente deployable correspondiente. Por ejemplo se suelen identificar tareas de relevamiento adicional, diseño de base de datos, diseño de interfaz gráfica, programación, armado de lotes de pruebas, pruebas, etc.

2. Para cada tarea se estima su duración en horas de esfuerzo. Si el esfuerzo resultante es mayor a un día de trabajo, se divide la tarea. Si el esfuerzo resultante es menor a una hora, se trata de agrupar con otras tareas.

3. Se realizan distintos chequeos para validar la factibilidad de cumplir con los objetivos de sprint (por ejemplo horas estimadas vs. horas disponibles del equipo). Si fuera necesario se acuerda un cambio al alcance del sprint con el Dueño de Producto.

4. Se arma el Tablero de Sprint si corresponde.

Finalmente se acuerdan fechas, horarios y logística de las Reuniones Diarias, de la Revisión y de la Retrospectiva del sprint.

- REUNIÓN DIARIA

1) ¿Cuándo?: Una vez por día, en un horario fijo a definir por el Equipo. Se recomienda la primera hora del día de trabajo.

2) Participantes: Scrum Master + Equipo + Dueño de Producto (opcional)

3) Objetivos: Que los miembros del Equipo de Desarrollo sincronicen sus tareas, comuniquen sus avances y reporten al Scrum Master cualquier impedimento que pueda frenar o bloquear su trabajo.

4) Duración: Menor a quince minutos

5) Artefactos Involucrados: Backlog de Sprint: se actualiza en la reunión

6) Dinámica

Se hacen las siguientes preguntas a cada miembro del equipo uno por uno:

- ¿Qué hiciste desde la última reunión diaria? _ Eventualmente se actualiza el estado, la asignación y el esfuerzo restante de las tareas involucradas

- ¿Qué vas a hacer hasta la próxima reunión diaria? _ Eventualmente se actualiza el estado, la asignación y el esfuerzo restante de las tareas involucradas

- ¿Hay algún obstáculo que frena o bloquea tu trabajo? _ El Scrum Master registra los impedimentos detectados para trabajar en su resolución.

Finalmente el Scrum Master suele actualizar el Diagrama de Burndown y chequear la factibilidad de completar los objetivos del sprint en el tiempo restante. Si no es el caso el Scrum Master debería disparar las acciones necesarias para revertir la situación.

- REVISIÓN

1) ¿Cuándo? El último día del sprint.

2) Participantes: Scrum Master + Equipo + Dueño de Producto + Otros Involucrados/Interesados en el Producto (opcional)

3) Objetivos

- Que los interesados en el producto puedan visualizar y/o experimentar las funcionalidades construidas durante el sprint.

- Que el Dueño de Producto pueda aceptar o rechazar dichas funcionalidades.

- Que los interesados en el producto puedan elaborar eventuales mejoras al producto en base a lo visto.

4) Duración: Típicamente una o dos horas para exponer todas las funcionalidades desarrolladas.

5) Artefactos Involucrados: Incremento de Funcionalidad Potencialmente Deployable

6) Dinámica

1. Breve presentación por el Equipo de Desarrollo de los ítems del Backlog de Producto que estaban comprometidos para este sprint y se identifican los que efectivamente se terminaron de desarrollar.
2. El equipo hace una demostración con el software andando de las funcionalidades desarrolladas. Se suele mostrar el software funcionando en un ambiente similar al de producción, como por ejemplo un ambiente específico de pruebas o de pre-producción.
3. Se contestan eventuales preguntas de los interesados y del Dueño de Producto sobre las funcionalidades presentadas. Cada interesado presente puede hacer comentarios, observaciones o críticas constructivas a la funcionalidad presentada. También se pueden sugerir mejoras o nuevas funcionalidades a incluir en el Backlog de Producto para futura priorización.
4. El Dueño de Producto formaliza las funcionalidades que no fueron terminadas o que no cumplen con las expectativas del negocio para que vuelvan al Backlog de Producto para futura priorización.
5. El Dueño de Producto puede pedir pasar a producción algunas funcionalidades terminadas (eventualmente junto con otras funcionalidades terminadas en sprints anteriores).
6. El Scrum Master anuncia la fecha y lugar de la próxima reunión de Revisión.

- RETROSPECTIVA

- 1) ¿Cuándo? Al finalizar el sprint, es la última actividad del sprint.
- 2) Participantes: Scrum Master + Equipo + Dueño de Producto (opcional)
- 3) Objetivos: Inspeccionar los procesos de trabajo y las interacciones que ocurrieron en el último sprint para detectar fortalezas del equipo y oportunidades de mejora. Se busca acordar unas pocas acciones concretas de mejora a implementar en el próximo sprint para hacerlo más productivo.
- 4) Duración: Se recomienda una duración de entre una y tres horas de retrospectiva (varía según la cantidad de personas y la duración del sprint).
- 5) Artefactos involucrados: No aplica.
- 6) Dinámica: Actividades grupales facilitadas por el Scrum Master los puntos siguientes:
 - ¿Qué mecanismos, procedimientos o interacciones han funcionado bien en este sprint?
 - ¿Qué mecanismos, procedimientos o interacciones hay que mejorar? Luego el equipo y el Scrum Master definen y acuerdan unas pocas acciones de mejoras concretas que pueda ejecutar el equipo en el próximo sprint. Finalmente se suele revisar el resultado de

las acciones de mejoras que se acordaron en retrospectivas anteriores para decidir si seguirlas, pararlas o adecuarlas.

PRÁCTICAS ADICIONALES

- REFINAMIENTO DEL BACKLOG

También conocido como Backlog Grooming, Backlog Refinement, Pre-Planning o Pre-Planificación, el Refinamiento del Backlog tiene como objetivo mejorar y preparar el Backlog de Producto para las próximas ceremonias de Planificación.

Suele ocurrir una o varias veces durante el Sprint, en general unos días antes de la próxima Planificación, según el estado del Backlog de Producto y las necesidades del Dueño de Producto.

Es una práctica muy importante que permite minimizar el tiempo de la planificación y mejorar su calidad.

Equipo y el Dueño de Producto trabajan juntos (con la facilitación del Scrum Master), ejecutando una a varias de las actividades siguientes:

- Depuración de elementos
- Ordenamiento de elementos
- Análisis de requisitos
- División de elementos grandes en varios elementos más pequeños
- Unión de varios elementos
- Estimación o re-estimación de elementos

- TABLERO DE SPRINT

- un tablero físico (por ejemplo con rotafolios, panel de corcho, pizarrón, etc.) expuesto en la sala donde trabaja el Equipo.

- logra una visibilidad importante sobre el avance del Sprint para todas las personas que están ubicados en este lugar o que pasan por la sala.

- presenta el Backlog de Sprint visualmente, usualmente con post-its o tarjetas de cartón o papel para representar ítems y tareas del Sprint

- se registra información básica de los elementos comprometidos (como por ejemplo título, estimación de esfuerzo, prioridad, criterios de aceptación) y de las tareas (como por ejemplo tarea, esfuerzo restante estimado, responsable).
- dividir el tablero en tres columnas, con los tres estados de tareas: Pendiente, En Curso, Terminado, en las cuales se ubican las tareas según el avance.



Definition of Ready (DoR) - Definición de “listo”

- Sirve para evaluar el trabajo antes de que el equipo comience a desarrollarlo.
- Define una tarea, historia de usuario o punto de historia, y permite actuar de inmediato si se trabaja con Scrum.
- A qué clientes se dirige: sus motivaciones, problemas y necesidades.
- Las tareas requeridas: si aportan valor para el negocio y el usuario, y si son claras y factibles.
- Los requisitos técnicos: recursos disponibles, enfoque o solución técnica y capacidad de prueba.

Las estimaciones de tiempo: plazos para completar el trabajo y acuerdo con los interesados.

Definition of Done (DoD) - Definición de “terminado”

- Conjunto de criterios que definen cuándo un incremento de producto se considera completo y listo para su entrega a los clientes.
- Entendimiento común entre todos los miembros del equipo.
- Tener criterios claros ayuda a enfocarse en la entrega de valor en cada sprint y a minimizar retrabajos.
- Toda la organización participa en la creación de la DoD, incluidos desarrolladores, testers, product owners y demás interesados.
- Se garantiza que todos utilicen la misma guía y checklist antes de marcar una tarea como finalizada